SQL JOIN and GROUP BY Lab - Student Exercises

Lab Introduction

This lab is designed to help you practice SQL JOINs and GROUP BY clauses through hands-on exercises. You will work with a company database containing information about departments, employees, customers, orders, products, and order details.

Database Schema Overview

The database contains the following tables:

departments

- dept_id (Primary Key)
- dept_name
- location

employees

- employee_id (Primary Key)
- first_name
- last_name
- email
- hire_date
- salary
- dept_id (Foreign Key → departments)
- manager_id (Foreign Key → employees)

customers

- customer_id (Primary Key)
- customer_name
- email
- city

- country

orders

- order_id (Primary Key)
- customer_id (Foreign Key → customers)
- order_date
- total_amount
- status

products

- product_id (Primary Key)
- product_name
- category
- unit_price

order_details

- order_detail_id (Primary Key)
- order_id (Foreign Key → orders)
- product_id (Foreign Key → products)
- quantity
- unit_price

Setup Instructions

Before starting the exercises, run the setup script provided by your trainer to create the database tables and insert sample data.

Setting up Databse

Query:

```
CREATE DATABASE sql_join_groupby_lab;
USE sql_join_groupby_lab;
CREATE TABLE departments (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50),
   location VARCHAR(50)
);
CREATE TABLE employees (
   employee_id INT PRIMARY KEY,
   first_name VARCHAR(50),
   last_name VARCHAR(50),
   email VARCHAR(100),
   hire_date DATE,
   salary DECIMAL(10,2),
   dept_id INT,
   manager_id INT,
   FOREIGN KEY (dept_id) REFERENCES departments(dept_id),
   FOREIGN KEY (manager_id) REFERENCES employees(employee_id)
);
CREATE TABLE customers (
   customer_id INT PRIMARY KEY,
   customer_name VARCHAR(100),
   email VARCHAR(100),
   city VARCHAR(50),
   country VARCHAR(50)
);
```

```
CREATE TABLE orders (
   order_id INT PRIMARY KEY,
   customer_id INT,
   order_date DATE,
   total_amount DECIMAL(10,2),
   status VARCHAR(20),
   FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
CREATE TABLE products (
   product_id INT PRIMARY KEY,
   product_name VARCHAR(100),
   category VARCHAR(50),
   unit_price DECIMAL(10,2)
);
CREATE TABLE order_details (
   order_detail_id INT PRIMARY KEY,
   order_id INT,
   product_id INT,
   quantity INT,
   unit_price DECIMAL(10,2),
   FOREIGN KEY (order_id) REFERENCES orders(order_id),
   FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Inserting data into Databse

Query:

```
INSERT INTO departments VALUES
(1,'HR','Hyderabad'),
(2,'IT','Bangalore'),
```

```
(3,'Finance','Mumbai'),
(4,'Sales','Delhi');
INSERT INTO employees VALUES
(1,'Amit','Sharma','amit@corp.com','2018-01-10',90000,2,NULL),
(2,'Neha','Verma','neha@corp.com','2019-03-15',70000,2,1),
(3,'Ravi','Kumar','ravi@corp.com','2020-06-20',60000,2,1),
(4,'Priya','Singh','priya@corp.com','2017-04-12',80000,3,NULL),
(5,'Karan','Mehta','karan@corp.com','2021-09-01',55000,1,4),
(6,'Anita','Rao','anita@corp.com','2016-11-25',95000,4,NULL);
INSERT INTO customers VALUES
(1,'Rahul Gupta','rahul@gmail.com','Delhi','India'),
(2,'Emily Clark','emily@gmail.com','London','UK'),
(3,'John Doe','john@gmail.com','New York','USA'),
(4,'Sita Devi','sita@gmail.com','Chennai','India');
INSERT INTO orders VALUES
(101,1,'2024-01-10',5000,'Completed'),
(102,1,'2024-02-15',3000,'Completed'),
(103,2,'2024-03-05',7000,'Pending'),
(104,3,'2024-01-20',4000,'Completed'),
(105,4,'2024-04-18',2000,'Completed');
INSERT INTO products VALUES
(1,'Laptop','Electronics',50000),
(2,'Mouse','Electronics',500),
(3,'Desk','Furniture',8000),
(4,'Chair','Furniture',4000),
(5,'Pen','Stationery',50);
INSERT INTO order_details VALUES
(1,101,1,1,50000),
(2,101,2,2,500),
```

(3,102,3,1,8000),

(4,104,4,2,4000),

(5,105,5,10,50);

Output:

Created Database and inserted data into the databse.

Exercises

Used chatgpt for syntax correctness.

Exercise 1: Basic INNER JOIN

List all employees with their department names. Display employee ID, first name, last name, and department name. Order the results by employee ID.

****Hint:**** You need to join the employees and departments tables.

****Expected columns:**** employee_id, first_name, last_name, dept_name

Query:

```
SELECT
   employees.employee_id,
   employees.first_name,
   employees.last_name,
   departments.dept_name
FROM employees
JOIN departments
ON employees.dept_id = departments.dept_id
ORDER BY employees.employee_id;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	first_name	last_name	dept_name
1	Amit	Sharma	IT
2	Neha	Verma	IT
3	Ravi	Kumar	IT
4	Priya	Singh	Finance
5	Karan	Mehta	HR
6	Anita	Rao	Sales

Used chatgpt for syntax correctness.

Exercise 2: LEFT JOIN to Find Missing Data

Find all customers and their order count, including customers who haven't placed any orders. Display customer ID, customer name, and the count of orders. Order by order count in descending order.

Hint: Use LEFT JOIN to include customers with no orders, and COUNT with GROUP BY.

Expected columns: customer_id, customer_name, order_count

Query:

```
SELECT
    customers.customer_id,
    customers.customer_name,
    COUNT(orders.order_id) AS order_count
FROM customers
LEFT JOIN orders
    ON customers.customer_id = orders.customer_id
GROUP BY customers.customer_id, customers.customer_name
ORDER BY order_count DESC;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	order_count
1	Rahul Gupta	2
2	Emily Clark	1
3	John Doe	1
4	Sita Devi	1

Used chatgpt for syntax correctness.

Exercise 3: Multiple JOINs

List all orders with customer names and the total number of items in each order. Display order ID, customer name, order date, total amount, count of different items, and sum of quantities. Order by order ID.

Hint: You need to join three tables: orders, customers, and order_details.

Expected columns: order_id, customer_name, order_date, total_amount, item_count, total_quantity

Query:

```
SELECT
   orders.order_id,
   customers.customer_name,
   orders.order_date,
   orders.total_amount,
   COUNT(DISTINCT order_details.product_id) AS item_count,
   SUM(order_details.quantity) AS total_quantity
FROM orders
JOIN customers
   ON orders.customer_id = customers.customer_id
JOIN order_details
   ON orders.order_id = order_details.order_id
GROUP BY
   orders.order_id,
   customers.customer_name,
   orders.order_date,
   orders.total_amount
ORDER BY orders.order_id;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_name	order_date	total_amount	item_count	total_quantity
101	Rahul Gupta	2024-01-10	5000.00	2	3
102	Rahul Gupta	2024-02-15	3000.00	1	1
104	John Doe	2024-01-20	4000.00	1	2
105	Sita Devi	2024-04-18	2000.00	1	10

Used chatgpt for syntax correctness and the reference for functions that can be used

Exercise 4: GROUP BY with Aggregates

Calculate the average salary by department. Display department name, count of employees, average salary, minimum salary, and maximum salary. Order by average salary in descending order.

Hint: Join departments and employees, then use GROUP BY with aggregate functions.

Expected columns: dept_name, employee_count, avg_salary, min_salary, max_salary

Query:

```
SELECT
   departments.dept_name,
   COUNT(employees.employee_id) AS employee_count,
   AVG(employees.salary) AS avg_salary,
   MIN(employees.salary) AS min_salary,
   MAX(employees.salary) AS max_salary
FROM departments
JOIN employees
   ON departments.dept_id = employees.dept_id
GROUP BY departments.dept_name
ORDER BY avg_salary DESC;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_name	employee_count	avg_salary	min_salary	max_salary
Sales	1	95000.000000	95000.00	95000.00
Finance	1	80000.000000	80000.00	80000.00
IT	3	73333.333333	60000.00	90000.00
HR	1	55000.000000	55000.00	55000.00

Used chatgpt for syntax correctness.

Exercise 5: HAVING Clause

Find departments with an average salary greater than 65000 and more than 2 employees. Display department name, employee count, and average salary. Order by average salary in descending order.

Hint: Use HAVING clause to filter groups after aggregation.

Expected columns: dept_name, employee_count, avg_salary

Query:

```
SELECT
   departments.dept_name,
   COUNT(employees.employee_id) AS employee_count,
   AVG(employees.salary) AS avg_salary
FROM departments
JOIN employees
   ON departments.dept_id = employees.dept_id
GROUP BY departments.dept_name
HAVING AVG(employees.salary) > 65000
  AND COUNT(employees.employee_id) > 2
ORDER BY avg_salary DESC;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_name	employee_count	avg_salary
IT	3	73333.333333

Used chatgpt for syntax correctness

Exercise 6: SELF JOIN

List all employees with their manager's name. Display employee ID, employee full name, employee salary, manager full name, and manager salary. Order by employee ID.

Hint: Join the employees table to itself. Use LEFT JOIN to include employees without managers.

Expected columns: employee_id, employee_name, employee_salary, manager_name, manager_salary

Query:

```
SELECT
    e1.employee_id,
    CONCAT(e1.first_name, ' ', e1.last_name) AS employee_name,
    e1.salary AS employee_salary,
    CONCAT(e2.first_name, ' ', e2.last_name) AS manager_name,
    e2.salary AS manager_salary
FROM employees e1
LEFT JOIN employees e2
    ON e1.manager_id = e2.employee_id
ORDER BY e1.employee_id;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	employee_salary	manager_name	manager_salary
1	Amit Sharma	90000.00	NULL	NULL
2	Neha Verma	70000.00	Amit Sharma	90000.00
3	Ravi Kumar	60000.00	Amit Sharma	90000.00
4	Priya Singh	80000.00	NULL	NULL
5	Karan Mehta	55000.00	Priya Singh	80000.00
6	Anita Rao	95000.00	NULL	NULL

Note: The null values here are expected and right outcome as we used self join in the query

Used chatgpt for syntax correctness.

Exercise 7: Complex Aggregation

Find the total revenue by product category. Display category, count of distinct orders, total units sold, and total revenue. Order by total revenue in descending order.

Hint: Join products and order_details, calculate revenue as quantity × unit_price.

Expected columns: category, order_count, total_units_sold, total_revenue

Query:

```
SELECT
   products.category,
   COUNT(DISTINCT order_details.order_id) AS order_count,
   SUM(order_details.quantity) AS total_units_sold,
   SUM(order_details.quantity * order_details.unit_price) AS total_revenue
FROM products
JOIN order_details
   ON products.product_id = order_details.product_id
GROUP BY products.category
ORDER BY total_revenue DESC;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	order_count	total_units_sold	total_revenue
Electronics	1	3	51000.00
Furniture	2	3	16000.00
Stationery	1	10	500.00

Used chatgpt for syntax correctness.

Exercise 8: JOINs with WHERE and HAVING

Find customers who have placed more than 1 completed order and their total spending. Display customer ID, customer name, count of completed orders, and total spent. Order by total spent in descending order.

Hint: Filter for completed orders using WHERE, then use HAVING to filter groups with more than 1 order.

Expected columns: customer_id, customer_name, completed_orders, total_spent

Query:

```
SELECT
    customers.customer_id,
    customers.customer_name,
    COUNT(orders.order_id) AS completed_orders,
    SUM(orders.total_amount) AS total_spent
FROM customers
JOIN orders
    ON customers.customer_id = orders.customer_id
WHERE orders.status = 'Completed'
GROUP BY customers.customer_id, customers.customer_name
HAVING COUNT(orders.order_id) > 1
ORDER BY total_spent DESC;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	completed_orders	total_spent
1	Rahul Gupta	2	8000.00

Used chatgpt for syntax correctness

Exercise 9: Finding Unmatched Records

Find all products that have never been ordered. Display product ID, product name, category, and unit price. Order by product ID.

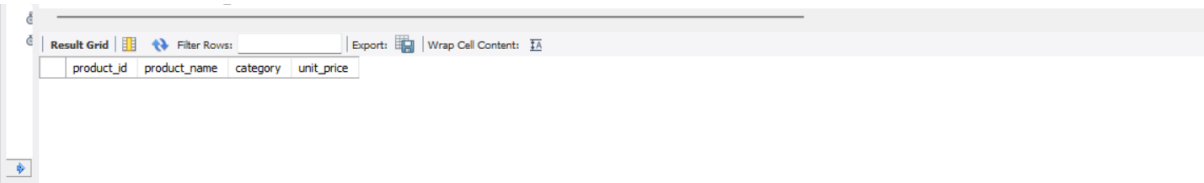
Hint: Use LEFT JOIN and check for NULL values in the joined table.

Expected columns: product_id, product_name, category, unit_price

Query:

```
SELECT
    products.product_id,
    products.product_name,
    products.category,
    products.unit_price
FROM products
LEFT JOIN order_details
    ON products.product_id = order_details.product_id
WHERE order_details.product_id IS NULL
ORDER BY products.product_id;
```

Output:



Note: The output is zero rows because there are no products that have never been ordered

Used chatgpt for syntax correctness

Exercise 10: Advanced Multi-Table Analysis

For each country, show the total number of orders, total revenue, and average order value. Only include countries that have orders. Display country, total orders, customer count, total revenue, and average order value. Order by total revenue in descending order.

Hint: Join customers and orders, group by country, and use HAVING to exclude countries with no orders.

Expected columns: country, total_orders, customer_count, total_revenue, avg_order_value

Query:

```
SELECT
    customers.country,
    COUNT(orders.order_id) AS total_orders,
    COUNT(DISTINCT customers.customer_id) AS customer_count,
    SUM(orders.total_amount) AS total_revenue,
    AVG(orders.total_amount) AS avg_order_value
FROM customers
JOIN orders
    ON customers.customer_id = orders.customer_id
GROUP BY customers.country
HAVING COUNT(orders.order_id) > 0
ORDER BY total_revenue DESC;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country	total_orders	customer_count	total_revenue	avg_order_value
India	3	2	10000.00	3333.333333
UK	1	1	7000.00	7000.000000
USA	1	1	4000.00	4000.000000

Used chatgpt for syntax correctness

Exercise 11: Subquery with GROUP BY

Find departments where the average salary is above the company average. Display department name, department average salary, and company average salary. Order by department average salary in descending order.

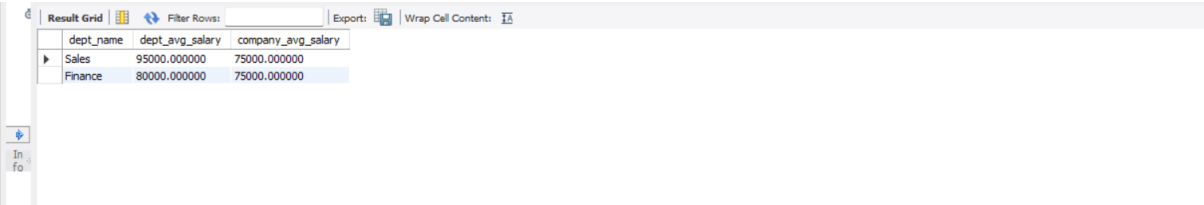
Hint: Use a subquery in the HAVING clause to compare against the overall average.

Expected columns: dept_name, dept_avg_salary, company_avg_salary

Query:

```
SELECT
   departments.dept_name,
   AVG(employees.salary) AS dept_avg_salary,
   (SELECT AVG(salary) FROM employees) AS company_avg_salary
FROM departments
JOIN employees
   ON departments.dept_id = employees.dept_id
GROUP BY departments.dept_name
HAVING AVG(employees.salary) >
      (SELECT AVG(salary) FROM employees)
ORDER BY dept_avg_salary DESC;
```

Output:



dept_name	dept_avg_salary	company_avg_salary
Sales	95000.000000	75000.000000
Finance	80000.000000	75000.000000

Used chatgpt for syntax correctness and for usage of ‘DATE_FORMAT’

Exercise 12: Date-Based Grouping

Show monthly order statistics for 2024. Display month (YYYY-MM format), order count, monthly revenue, and average order value. Order by month.

Hint: Use date functions to extract year-month. Functions vary by database:

- SQLite: `strftime('%Y-%m', order_date)`

- MySQL: `DATE_FORMAT(order_date, '%Y-%m')`

- PostgreSQL: `TO_CHAR(order_date, 'YYYY-MM')`

Expected columns: month, order_count, monthly_revenue, avg_order_value

Query:

```
SELECT
   DATE_FORMAT(order_date, '%Y-%m') AS month,
   COUNT(order_id) AS order_count,
   SUM(total_amount) AS monthly_revenue,
   AVG(total_amount) AS avg_order_value
FROM orders
WHERE YEAR(order_date) = 2024
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY month;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	order_count	monthly_revenue	avg_order_value
2024-01	2	9000.00	4500.000000
2024-02	1	3000.00	3000.000000
2024-03	1	7000.00	7000.000000
2024-04	1	2000.00	2000.000000

Used chatgpt for syntax correctness

Exercise 13: Complex Filtering

Find the top 3 products by revenue. Display product ID, product name, category, total quantity sold, total revenue, and times ordered. Order by total revenue in descending order and limit to 3 results.

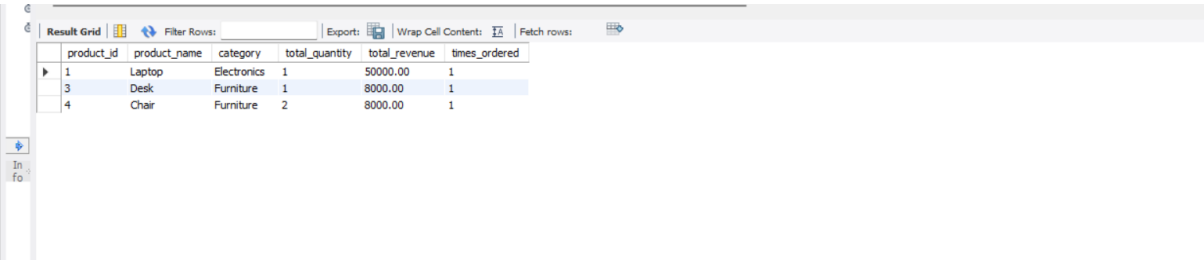
Hint: Calculate revenue as quantity × unit_price, use ORDER BY and LIMIT.

Expected columns: product_id, product_name, category, total_quantity, total_revenue, times_ordered

Query:

```
SELECT
  products.product_id,
  products.product_name,
  products.category,
  SUM(order_details.quantity) AS total_quantity,
  SUM(order_details.quantity * order_details.unit_price) AS total_revenue,
  COUNT(DISTINCT order_details.order_id) AS times_ordered
FROM products
JOIN order_details
  ON products.product_id = order_details.product_id
GROUP BY
  products.product_id,
  products.product_name,
  products.category
ORDER BY total_revenue DESC
LIMIT 3;
```

Output:



product_id	product_name	category	total_quantity	total_revenue	times_ordered
1	Laptop	Electronics	1	50000.00	1
3	Desk	Furniture	1	8000.00	1
4	Chair	Furniture	2	8000.00	1

Used chatgpt for syntax correctness and for using CASE

Exercise 14: CASE with GROUP BY

Categorize employees by salary range and count employees in each range by department. Salary ranges:

- Entry Level: Less than 60000

- Mid Level: 60000 to 74999

- Senior Level: 75000 and above

Display department name, salary range, employee count, and average salary. Order by department name and then average salary.

****Hint:**** Use CASE statement to create salary ranges, include it in both SELECT and GROUP BY.

****Expected columns:**** dept_name, salary_range, employee_count, avg_salary

Query:

```
SELECT
  departments.dept_name,
  CASE
    WHEN employees.salary < 60000 THEN 'Entry Level'
```

```
        WHEN employees.salary BETWEEN 60000 AND 74999 THEN 'Mid Level'
        ELSE 'Senior Level'
    END AS salary_range,
    COUNT(employees.employee_id) AS employee_count,
    AVG(employees.salary) AS avg_salary
FROM departments
JOIN employees
    ON departments.dept_id = employees.dept_id
GROUP BY
    departments.dept_name,
    CASE
        WHEN employees.salary < 60000 THEN 'Entry Level'
        WHEN employees.salary BETWEEN 60000 AND 74999 THEN 'Mid Level'
        ELSE 'Senior Level'
    END
ORDER BY departments.dept_name, avg_salary;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

dept_name	salary_range	employee_count	avg_salary
Finance	Senior Level	1	80000.000000
HR	Entry Level	1	55000.000000
IT	Mid Level	2	65000.000000
IT	Senior Level	1	90000.000000
Sales	Senior Level	1	95000.000000

Used chatgpt for syntax correctness

Exercise 15: Comprehensive Report

Create a comprehensive customer report for customers with completed orders. Display: customer ID, customer name, country, total orders, total spent, average order value, last order date, and first order date. Order by total spent in descending order.

Hint: Join customers and orders, filter for completed status, use multiple aggregate functions.

Expected columns: customer_id, customer_name, country, total_orders, total_spent, avg_order_value, last_order_date, first_order_date

Query:

```
SELECT
    customers.customer_id,
    customers.customer_name,
    customers.country,
    COUNT(orders.order_id) AS total_orders,
    SUM(orders.total_amount) AS total_spent,
    AVG(orders.total_amount) AS avg_order_value,
    MAX(orders.order_date) AS last_order_date,
    MIN(orders.order_date) AS first_order_date
FROM customers
JOIN orders
    ON customers.customer_id = orders.customer_id
WHERE orders.status = 'Completed'
GROUP BY
    customers.customer_id,
    customers.customer_name,
    customers.country
ORDER BY total_spent DESC;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	country	total_orders	total_spent	avg_order_value	last_order_date	first_order_date
1	Rahul Gupta	India	2	8000.00	4000.000000	2024-02-15	2024-01-10
3	John Doe	USA	1	4000.00	4000.000000	2024-01-20	2024-01-20
4	Sita Devi	India	1	2000.00	2000.000000	2024-04-18	2024-04-18

Used chatgpt for syntax correctness

Challenge Exercises (Bonus)

These exercises are optional and more advanced. Attempt them if you finish the main exercises early.

Used chatgpt for syntax correctness and for working with CTE and RECURSIVE

Challenge 1: Employee Hierarchy Analysis

Show each employee's level in the organization hierarchy and their path from the top. This requires a recursive query.

Hint: Use a recursive CTE (Common Table Expression) with WITH RECURSIVE.

Expected columns: employee_id, employee_name, manager_id, level, path

Query:

```
WITH RECURSIVE employee_hierarchy AS (
  SELECT
    employee_id,
    CONCAT(first_name, ' ', last_name) AS employee_name,
    manager_id,
    1 AS level,
    CONCAT(first_name, ' ', last_name) AS path
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT
    employees.employee_id,
    CONCAT(employees.first_name, ' ', employees.last_name) AS employee_name,
    employees.manager_id,
    employee_hierarchy.level + 1 AS level,
```

```
        CONCAT(employee_hierarchy.path, ' → ',
            employees.first_name, ' ', employees.last_name) AS path
    FROM employees
    JOIN employee_hierarchy
        ON employees.manager_id = employee_hierarchy.employee_id
)
SELECT
    employee_id,
    employee_name,
    manager_id,
    level,
    path
FROM employee_hierarchy
ORDER BY path;
```

Output:

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	manager_id	level	path
1	Amit Sharma	NULL	1	Amit Sharma
2	Neha Verma	1	2	Amit Sharma → Neha Verma
3	Ravi Kumar	1	2	Amit Sharma → Ravi Kumar
6	Anita Rao	NULL	1	Anita Rao
4	Priya Singh	NULL	1	Priya Singh
5	Karan Mehta	4	2	Priya Singh → Karan Mehta

Used chatgpt for syntax correctness

Challenge 2: Product Affinity Analysis

Find products that are frequently bought together (appear in the same order at least twice). Display the two product names and how many times they were bought together.

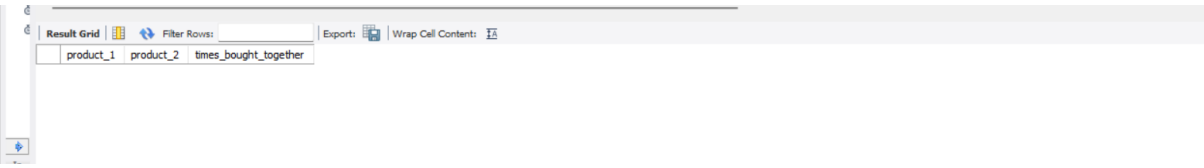
****Hint:**** Self-join order_details on order_id, ensure product_id1 < product_id2 to avoid duplicates.

Expected columns: product_1, product_2, times_bought_together

Query:

```
SELECT
   p1.product_name AS product_1,
   p2.product_name AS product_2,
   COUNT(*) AS times_bought_together
FROM order_details od1
JOIN order_details od2
   ON od1.order_id = od2.order_id
  AND od1.product_id < od2.product_id
JOIN products p1
   ON od1.product_id = p1.product_id
JOIN products p2
   ON od2.product_id = p2.product_id
GROUP BY p1.product_name, p2.product_name
HAVING COUNT(*) >= 2
ORDER BY times_bought_together DESC;
```

Output:



Note: The output is zero rows because there are no products that appear together in the same order at least twice

Tips for Success

1. **Read the requirements carefully** - Make sure you understand what data is requested
2. **Start with the FROM clause** - Identify which tables you need
3. **Add JOINs one at a time** - Build your query incrementally
4. **Test your query** - Run it to see if you get the expected columns
5. **Check your results** - Do the numbers make sense?
6. **Use table aliases** - Makes queries more readable (e.g., `FROM employees e`)
7. **Format your SQL** - Use proper indentation and line breaks
8. **Comment your code** - Add comments to explain complex logic

Common Mistakes to Avoid

- Forgetting to include columns in GROUP BY that appear in SELECT
- Using WHERE instead of HAVING for filtering aggregated results
- Incorrect JOIN conditions leading to Cartesian products
- Not using LEFT JOIN when you need to include unmatched records
- Forgetting to order results when specified

Submission Guidelines

1. Save your queries in a file named `firstname_lastname_sql_lab.sql`
2. Include comments with your name and the exercise number
3. Format your SQL code properly with indentation
4. Test all queries before submission
5. Submit your file to your trainer

Evaluation Criteria

Your work will be evaluated on:

- **Correctness (60%)** - Does the query return the expected results?
- **Query Structure (20%)** - Proper use of JOINs, GROUP BY, and HAVING?
- **Efficiency (10%)** - Is the query reasonably optimized?
- **Code Quality (10%)** - Is the SQL clean, readable, and well-formatted?

Good luck with your exercises!